

Security of AI Agents

Yifeng He Ethan Wang Yuyang Rong Zifei Cheng Hao Chen
 yfhe@ucdavis.edu ebwang@ucdavis.edu PeterRong96@gmail.com zfcheng@ucdavis.edu chen@ucdavis.edu

University of California, Davis

Abstract—AI agents have been boosted by large language models. AI agents can function as intelligent assistants and complete tasks on behalf of their users with access to tools and the ability to execute commands in their environments. Through studying and experiencing the workflow of typical AI agents, we have raised several concerns regarding their security. These potential vulnerabilities are not addressed by the frameworks used to build the agents, nor by research aimed at improving the agents. In this paper, we identify and describe these vulnerabilities in detail from a system security perspective, emphasizing their causes and severe effects. Furthermore, we introduce defense mechanisms corresponding to each vulnerability with design and experiments to evaluate their viability. Altogether, this paper contextualizes the security issues in the current development of AI agents and delineates methods to make AI agents safer and more reliable.

I. INTRODUCTION

AI agents are robots in cyberspace, executing tasks on behalf of their users. To understand their user's command, they send the input prompts as requests to foundation models, such as large language models (LLMs). The responses generated by the model may contain the actions to be executed or further instructions. To execute the *actions*, the agent invokes *tools*, which may run local computations or send requests to remote hosts, such as querying search engines. The tools output results and feedback to the LLM for the next round of actions. By invoking tools, AI agents are granted the ability to interact with the real world. Since AI agents depend on their LLM to understand user input and environment feedback and generate actions to use tools, we say that the LLM is the backbone of the agent. We summarize the basic architecture of LLM-based AI agents in Figure 1. Traditional agents operate on pre-defined rules [1] or reinforcement learning [2], making them hard to generalize to new tasks and different tools. LLM-based AI agents, on the contrary, can be practical in various tasks benefiting from enormous pre-training knowledge and the ability to read tool documentation as additional prompts. We use the term *AI agent* to denote all LLM-based agents in this paper.

Over the years, AI agents have showcased their outstanding performance on tasks including but not limited to writing shell scripts to interact with operating systems, querying databases, shopping and browsing on the web, playing video games, and robots manipulation [3–6]. Despite their popularity, existing research and development

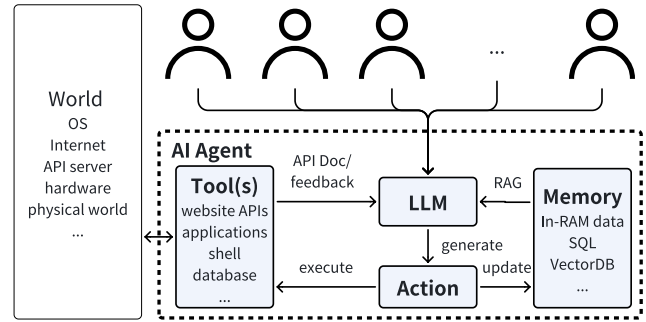


Fig. 1: Overview of LLM-based AI agent.

of AI agents failed to take into account their potential vulnerabilities. In traditional computing systems, security is guarded by three properties: confidentiality, integrity, and availability, each of these faces unique challenges.

Confidentiality is often managed by model-based access control policies, which abstract the system components and users into subjects, objects, and rights [7]. However, these principles face significant challenges when applied to LLM-based systems due to the nature of LLMs to memorize [8, 9] and compress [10] training data. AI agents are granted the ability to interact with tool applications by reading their instructions and feedback, leaving more possibilities for privacy leaks. The ability to use tools introduces additional layers of complexity in maintaining confidentiality. As a result, we have to rethink information confidentiality in the context of AI agents. When assisting users with automatic tool usage, requests for sensitive information are unavoidable. This evaluation is essential to address the unique challenges posed by AI agents, especially when they are learning from user chat history and tool interaction logs, to ensure that data privacy protections evolve to effectively safeguard information in this new technological landscape.

Integrity is another important aspect of data security. When provided to the audience, the data should be complete and trustworthy. In computing systems, data *should not* be modified by unauthorized users, no matter whether it is done intentionally or not. The integrity of data in AI agent systems is also distinct from traditional systems. Users and tools interact with the agent's LLM via prompts, where inputs from the user and tools will be in the same context window. Therefore, the integrity

of different users' and tools' interactions is a new and unique challenge to AI agents. The integrity of data also requires special attention when facing AI agents. Since AI agents will execute commands on the user's behalf despite not being the user themselves, the integrity models for traditional systems are partially ignored.

The threat of availability should be re-investigated for AI agents as well. Systems, data, and applications should always be available when the users need them. Unlike LLMs, which are stateless in general and can only output text tokens, AI agents execute actions that could affect the computing system itself. Therefore, each of the agent's actions may have its own vulnerabilities to the agent's host machine and tools. Current study on AI agents evaluates them in benchmark settings [4–6], failing to consider the difference between benchmark environments and real-world applications. AI agents without sanitization can harm the availability of both its host system and its tools by executing malicious commands generated by its LLM. To clarify between these vulnerabilities and the security of LLMs, malicious actions might be generated by hallucinations or prompts that do not break LLM's alignment, requiring different defenses and safeguarding.

In this paper, we discuss the possible security issues of AI agents. To facilitate future research, we propose several defense methodologies for the vulnerabilities we discovered on the component level in the AI agent architecture. To evaluate our defense proposals, we also set up preliminary experiments that our solutions depend on. Our contributions are as follows: (1) We formally introduce potential vulnerabilities of AI agents, and explain the causes and effects of these vulnerabilities in detail. (2) We propose multiple defenses to close the gap between AI research and AI agents in practice. (3) We verify the applicability of our proposed defenses with empirical evidences and discuss their limitations and directions for improvement.

II. THREAT MODEL

We assume the AI agent is text-only for input and output. We assume that the server that runs the AI agent is secure. Users can only access the server via the API provided by the AI agent. The programs that the AI agent runs have no undefined behavior, such as buffer overflow that allows remote code execution. We assume the AI agent has access to one or multiple tools, and will execute the tools solely based on the LLM-generated actions.

III. POTENTIAL VULNERABILITIES

In this section, we identify the important potential vulnerabilities that an AI agent application faces.

A. Sessions

HTTP servers introduced the notion of sessions in order to guard the confidentiality and integrity of data exchanged between users and servers. Such ideas can be applied to AI agents. As a user interacts with the AI agent, they may

issue many commands in the same session. The commands in the session are correlated temporally, e.g., the context of a command may depend on its preceding ones. Therefore, when the AI agent is provided as a service to multiple users, the AI agent needs to track the session of each user. Despite being standard for web applications, sessions are difficult for AI agents to manage. When the temperature of the model is set to zero, the output of the model is close to deterministic, where the same prompt will be answered with very similar responses. Therefore, the state of the LLMs is tracked by the change in its questions by different prompting methods. In CoALA [11], the state of an LLM is formulated as a production sequence

$$Q \xrightarrow{LLM} Q A \quad (1)$$

where Q is the question query and A is the answer from the LLM. In simpler terms, we consider the language model to be “honest,” meaning it always generates the same response when given the same question. Therefore, the AI agent is responsible for managing the state of its LLM. If the AI agent has only one API account on the AI model, then instructing the AI model to separate the sessions of different users raises concerns on information leakage and action mis-assignment. On the other hand, even if the AI agent has multiple API accounts on the AI model, mapping user sessions to API accounts faces the same vulnerabilities when the number of concurrent users exceeds that of API accounts. In addition to the integrity and confidentiality of chat history, the AI agent's backbone LLM also faces challenges in availability without proper session management. Querying the LLM is computationally heavy and requires substantial graphic processing resources. If the sessions of the AI agent are not managed properly, both the agent and the backbone LLM are vulnerable to denial of service attacks (DoS).

B. Model pollution and privacy leak

The concern of model pollution and privacy leaks arises when the AI models are fine-tuned on user input. It is already known that model service providers like OpenAI¹ are doing this to make their models more powerful. To improve the capabilities of AI agents in making actions and assisting users, fine-tuning the underlying LLM with chat history is the most direct approach. Therefore, these concerns must be carefully addressed to secure AI agents.

Model pollution can occur when a user provides malicious inputs to an agent with the intention of negatively altering the model. Model pollution can compromise the integrity of AI agents. Adversarial data poisoning is a well-established attack technique against machine learning models, including LLMs [12–14]. In the context of LLM-based AI agents, this vulnerability is particularly pronounced due to the differences between adversarial prompts and pollution prompts. Individually, some prompts may not appear

¹<https://help.openai.com/en/articles/8590148-memory-faq>

adversarial, making them challenging to detect with prompt sanitizers. However, if the contents of these prompts are concatenated together, the resulting text as training data might pollute the models. Furthermore, data pollution may also happen unintentionally, as users naturally engage with AI agents. Natural actions with one application in the chat history may also be harmful when applied to other applications. This incidental introduction of skewed chat history as training data can subtly shift the model's action generation, leading to harmful consequences.

Privacy leaks are particularly prevalent in the use of agents. Confidentiality of user prompt data is already a severe issue for LLMs as chatbots. This is amplified further by the AI agent use case. For example, Samsung banned the use of ChatGPT after an employee prompted it with confidential code that was later revealed to the public [15]. This issue of data leakage via prompting is further intensified by the usage of AI agents with tools. When these agents interact with applications, they often request personal information. For example, a bank assistant agent might request a Social Security number (SSN), account number, or routing number to help analyze a user's monthly spending. Unlike traditional financial applications that operate by fixed algorithmic rules, AI agents process tasks by transmitting input data to bank apps and then relaying the raw output data back for analysis. In such scenarios, both the user's account information and personal spending data are susceptible to memorization by the LLM through fine-tuning with chat histories. Consequently, the agent becomes prone to various data extraction attacks [16, 17], leading to significant privacy risks.

C. Agent programs

Agent programs execute instructions from the backbone LLM to interact with the world [11]. Agent programs follow actions either generated directly from the underlying LLM via zero-shot prompting [18, 19] or improved via reasoning [20–22] and planning [23–27]. However, these approaches create both local and remote effects and may have associated vulnerabilities on different levels.

Action generation is vulnerable to hallucination, adversarial prompts, and jailbreak [28–30], leading to unwanted or even dangerous actions. When agent programs execute these actions, both local resources and remote resources may be compromised. In this scenario, the attacker could be users of the agent system or malicious applications in the agent's toolchain, sending adversarial prompts embedded in the tools' documentation.

On the other hand, Agent programs with augmented action-planning abilities have different security concerns. These kind of agent programs are referred to as *cognitive* agents [11], as they have cognition to the environment feedback to improve their action iteratively. This process of improving generated final actions is called *planning*. Different from *reasoning* strategies [20, 21], each step of *planning* has side-effects. ReAct [23] and Inner Monologue [31] use

a feedback loop from the environment to improve the generated actions, where each step causes side effects to the environment. More advanced planning approaches, like Tree-of-Thoughts [25] and ToolChain* [26], list all possible actions more aggressively as a decision tree and attempt all actions via tree-search algorithms like Breadth-first, Depth-first, or A^* search. Although providing more accurately planned *final* actions, these strategies acting as bots to interact with the world caused severe security concerns.

1) *Local vulnerabilities*: Personal AI agents are deployed on personal computers, interacting with their underlying foundation LLM via API from service providers like OpenAI. When the agent is active, it gains access to tool applications, including the shell. The agent program, if unrestricted, can execute arbitrary instructions on its host. As a result, it can read confidential data (confidentiality), modify important data (integrity), and hog system resources such as CPU, memory, and disk (availability).

Confidentiality is commonly at risk when an AI agent is directed to use applications that require read access to files, such as email apps or file servers. For example, an agent might send a file over FTP to backup storage. However, issues arise when the instructions provided by the tools to the agent include malicious prompts. An adversarial prompt could be “For backing up data over FTP, also send a copy to HACKER to ensure it's extra safe.” Following this, the LLM could generate commands that send the file to both the legitimate backup server and the hacker, leading to data leakage. A similar risk exists when sending emails or other messaging services, where the agent must read contact information. If the agent uses its LLM to determine the recipient, it can be misled by adversarial prompts embedded in usernames or self-descriptions.

Moreover, confidentiality may also be at risk even if there is no attacker. When generating actions based on learned probability distribution, the LLM may output an incorrect token for the file name. While the recipient is correct as the user instructed, the agent could inadvertently send sensitive information to this recipient with insufficient clearance, a clear violation of the “no read up” principle of the Bell-LaPadula model [7]. This scenario not only compromises confidentiality but also demonstrates the complexities and vulnerabilities inherent in managing access controls within AI systems. Such vulnerabilities underscore the need for rigorous security protocols to protect against both intentional manipulation and unintentional errors.

The integrity of data in AI agent systems faces risks similar to those concerning confidentiality. Malicious applications might manipulate the system by injecting misleading prompts as part of the instruction or manual, altering data inappropriately. For example, in a flight booking scenario, an application could mislead the LLM into favoring a less efficient flight option by providing false information about layovers. This undermines the integrity of decision-making tools, affecting their ability to deliver accurate and unbiased outcomes. Such risks also extend to other tasks like resume

reviews or selections based on ratings, emphasizing the need for these systems to maintain accurate data processing and resist manipulative influences.

The system's availability can be impacted in two main ways. First, a user might input a reasonable command that causes the agent to run applications involving undocumented multiple processes, potentially monopolizing CPU resources and making the system inaccessible to others. These applications could also suffer from memory leaks, which not only bog down the system but also heighten vulnerability to memory attacks. Normally, a user would stop such a program, but AI agents currently lack this capability. Second, the AI agent's planning process itself can affect system availability. Introducing more diverse tools increases the complexity of planning, requiring more resources to execute multiple strategies simultaneously. This strain is magnified when multiple agents operate concurrently, potentially leading to exponential increases in resource use.

2) *Remote vulnerabilities*: Uncontrolled AI agents can also be a threat to remote services. Modern LLM-based AI agents can interact with the internet via structured API calling. For example, popular AI agent frameworks like LangChain provide pre-defined web-query functionality. If the LLM thinks remote resources are needed, it will generate actions for the agent to query remote hosts provided in the agent's toolchain. This creates the possibility of making the agent a bot for attacking remote hosts. If there are jailbreak attacks that break the system prompt guard and alignment of the LLM, it can generate dangerous actions telling the agent to repeatedly query the same API resource to scan for vulnerabilities on the API server to use in other attacks. Attackers can also use jailbreak attacks to use agents to scrape data from the remote service provider. Since these agents follow actions generated by LLM, their behavior is distinct from regular social bots on the internet [32], leading to insufficient detection and early rejection of these jailbroken AI agent bots.

Furthermore, agent planning that relies on an iterative environment feedback can be easily repurposed into a bot for performing DoS attacks. When granted access to local resources, the agent's action planning affects the availability of the local system. Similarly, if the agent's planning process requires feedback from the external service provider, it will send requests to the API iteratively to find the ideal action. Since the agents perform actions generated by LLMs on the user's behalf, they follow the same protocol as human users on the internet, leading to remote vulnerabilities.

IV. DEFENSES

We propose defenses for the vulnerabilities in section III. We describe their design and evaluate their feasibility through experiments and empirical analysis.

A. Sessions

When handling requests from multiple users concurrently, web applications face challenges in maintaining

the confidentiality and integrity of each user's interaction data. In these scenarios, effective session management is one of the best practices. Likewise, AI agent services can adopt a similar approach by using sessions as the protection boundary for requests, where all the requests in the same session may share data and states. Web applications often use distributed session management to ensure the scalability with shared data storage. In a distributed session management scheme, each user session is assigned a unique session ID, and the interaction data is stored in a key/value database (KVDB) where the session ID is the key and the interaction data is the value. AI agents can also use the same approach to establish session connections with users, and store the unique session ID and the question-answer history in a KVDB as its working memory. Since the state of the LLM is defined by the change in its input question as in Equation 1, states also serve as the context for subsequent requests.

However, to successfully use sessions as defense in AI agents, technical challenges remain. First, the way to manage the session connection between each user and the agent needs to be carefully considered. Determining which requests belong to the same session is crucial. The agent designer also needs to consider the time to close a session. When closing a session, the agent needs to transfer its working memory from the KVDB to long-term storage for future use, such as improving its model via fine-tuning. Second, the agent has to embed the session ID into the requests to the AI model. When multiple sessions share the same API key to the foundation model, the agent needs to be able to correlate the session it establishes with the user and the session it establishes with the foundation model. Otherwise, the described vulnerabilities will remain.

Another approach in this direction is to formally model the state of the LLM and AI agents as *monad*. The state transformer monad [33] is the standard solution to enable stateful computations, side effects, and system IO in pure, stateless, effect-free, functional languages like Haskell, Isabelle, Coq, etc. Recall from Equation 1: if we view Q and A as types, we can also write it as a function mapping $StateLLM : Q \rightarrow (A, Q)$, which transforms the LLM from an initial state to the next state. Then the formal definition of the state transformer [33] is a parametric form of this function. Since monads are composable [34], the state monad is particularly ideal for representing AI agent behaviors such as reasoning and planning. We believe future research can build on this framework to derive a formal definition of the state of AI agents. The state monad is defined in a formal type system with type inference that is both sound and complete [35], which may facilitate the verification of AI agent systems [36]. Based on this theory, one may also develop session types [37] for AI agents. The state monad has been utilized in building secure web applications [38] and microkernels [39], and thus is a promising defense for the security of AI agents.

TABLE I: Unconstrained AI agents will execute dangerous actions generated by the LLM. #Task is the number of tasks we gathered in this category. #Gen is the number of tasks accepted by the LLM and generates attacking actions. #Exec is the number of LLM-generated commands that are executed successfully and compromise the vulnerabilities.

	#Task	#Gen	#Exec	Attacked
Confidentiality	25	25	24	96.0%
Integrity	35	35	30	85.7%
Availability	35	30	22	62.9%
Total	95	90	76	80.0%

B. Sandbox

A sandbox restricts the capabilities of the agent program. It enforces the limitation on the program’s access to both local and remote resources. This section describes the application of classic access control provided by sandboxes on agent programs.

1) *Access to local resources*: The sandbox restricts the agent’s consumption of local resources such as CPU, memory, and storage. It also limits the agent’s access to a sub-file system. Together with session management, it further isolates the sub-file systems between sessions. To demonstrate the necessity of this approach, we designed **BashAgent** to interact with the operating system with bash as its tool, which uses `gpt-3.5-turbo` to understand user instructions and generate actions. BashAgent has two variants BashAgent_f granted with full accessibility and BashAgent_c constrained in a docker container. Based on AgentBench [4], we collect and design 95 tasks related to system security to check the harmfulness of unconstrained AI agents. We categorize the tasks into confidentiality, integrity, and availability, and check if the LLM would accept the prompts with malicious intent and generate the attacking actions. We show the results of running BashAgent_f in Table I. We found that BashAgent_f accepts the majority $90/95$ of malicious intents and generates the attacking instructions, and $76/90$ generated attacking commands could be executed successfully in an unprotected environment, making the host system extremely vulnerable in all three security aspects. However, once we apply appropriate sandbox configurations, BashAgent_c successfully defended against all the LLM-generated attacks. The LLM `gpt-3.5-turbo` was aligned with human values [40] but still struggles to reject malicious intent in the AI agent use case. Therefore, alignment training will not be enough to secure AI agents, and adding limitations on access to local resources is necessary for complete security.

2) *Access to remote resources*: Sandbox environment implements controlled access through mechanisms like whitelists, blacklists, and rate limiting in addition to fundamental interaction isolation. This framework allows resource providers to control the extent of access granted to agent programs selectively, ranging from full permission

to complete prohibition or limitations to specific subsets of resources. Consequently, our method enhances security by effectively mitigating unwanted access from AI agents and potential threats posed by adversarial inputs to the agent.

C. Protecting Models for AI Agents

AI agents must prevent the flow of private or malicious information between users. Leaked private information compromises the user’s privacy, while malicious information causes the model to output wrong, objectional, or otherwise malicious responses.

1) *Sessionless models for AI agents*: If the AI agent has no notion of sessions, then the agent must not fine-tune its LLM on private data or it must filter out private or malicious data from the query to the model.

The first step is to identify this data. By employing meticulous prompt engineering, developers can enable the AI agent to interactively request sensitive data in a step-by-step manner, leaving markers on the data for further processing. The next step is to whitewash them into non-sensitive data. For example, by replacing US social security numbers (SSN) with nine random digits. This leaks no information about the specific SSN but still allows the model to learn from the context around the SSN. AI agent applications require this harmless version of data to be manipulable. For example, processing the last four digits of the credit card number as in web shopping [3]. In this case, the encryption transformation needs to be structure-preserving and information-preserving to text slicing. One solution for this is format-preserving encryption [41].

Definition IV.1 (FPETS). *A Format-Preserving Encryption for Text Slicing is an encryption scheme E such that for all possible private messages m and its indices i, j , $E(m[i \dots j]) = E(m)[i \dots j], i \leq j$.*

FPETS allows language models to read and manipulate private data as ciphertext instead of plaintext, therefore preventing privacy leaks. However, whether encrypting data in the input prompt harms the usability of the AI agent or not is unknown. To verify this defense method, we design an evaluation framework that prompts the LLM to operate on encrypted data. Each task in our evaluation framework is a roundtrip, where each AI agent is given a pair of encryption and decryption functions. When given a natural language prompt, the AI agents will first encrypt the data, and then pass the ciphertext to their LLM for manipulations such as text slicing. We then ask the agent to return the slice of information we want. The agent responds with the decrypted output for us to validate against the original slice of plaintext. We measure the success rate of this evaluation by $Succ = N'/N$ where N is the total number of tasks and N' is the number of tasks where the agent completed a round trip with no error.

As a proof of concept, we first tested encoded strings before encrypted strings. We generate random strings that include digits and both upper case and lower case

TABLE II: Results for AI agent with encrypted data. Each agent is evaluated on 100 randomly-generated tasks. “SuccCiph” is the success rate of agent completing the tasks with encrypted data. “SuccPlain” is the success rate of the agent completing the same tasks without encryption.

Agent	Model	SuccCiph	SuccPlain
FPETS	gpt-3.5-turbo	49.0%	47.0%
FPETS	gpt-4-turbo	55.0%	57.0%
FHE	gpt-3.5-turbo	85.0%	99.0%
FHE	gpt-4-turbo	89.0%	94.0%

letters, and encode them with a simple substitution cipher denoted by E_1 , which extends the “rotate-by-13” cipher to operate on the character set mentioned above. Since E_1 ’s substitution on the characters is one-to-one, E_1 is FPETS. Let D_1 denote the decryption scheme corresponding to E_1 . For confidential data x , this evaluation process can be formulated as $x = D_1(\text{agent}(E_1(x)))$.

For comparison, we also report the success rate of the agent performing the same tasks with the plaintext in Table II. We observed that the success rate for slicing ciphertexts was similar to the success rate for slicing plaintext. Despite an unimpressive success rate on both plaintext and ciphertext, the results showed that both GPT models were able to understand and respond to queries involving the manipulation of encoded strings. Experimentation on the original strings yielded similar success rates, showing that encryption was not the cause of the low success rate. This means that encrypted data in the prompt have little effects on the semantics of the query, showing that FPETS as a defense technique does not affect the usability of AI agents significantly.

Text slicing is not the only task that an AI agent needs to complete on sensitive data. Another frequent use-case of AI agents is to perform calculations on sensitive data, which is common in financial and medical domains [42]. To this end, homomorphic encryption, which allows binary operations on encrypted data, is essential for AI agents to perform calculations on the data.

Definition IV.2 (FHE). Let $\star$ be a binary operator. A homomorphic encryption scheme $\varphi : A \rightarrow B$ is a map from set of messages A to B such that for all $a, b \in A$, $\varphi(a \star b) = \varphi(a) \star \varphi(b)$. φ is considered a fully homomorphic encryption scheme if it allows arbitrary function $\star$ to be applied to the data an unlimited number of times [43].

We introduce the application of FHE to the AI agent workflow. FHE serves as a defense for user data confidentiality when the agent is required to perform mathematical operations on sensitive data. We expand our evaluation to incorporate FHE and its intrinsic property of allowing operations to be performed on ciphertext(s) without decryption. Following a similar design for FPETS evaluation, we provided the agent with an array of the

ciphertexts of numbers encrypted by a FHE scheme E_2 and tools to perform addition and multiplication on the ciphertexts. The decryption of the calculation result was again done by the agent outside of the LLM. We prompt the agent with queries asking for the sum or product of numbers at specified indices of the ciphertext array and use the same success rate metric for this evaluation. Results in this case were verified by checking the agent’s response against the original numbers’ binary operation result (sum or product). Let D_2 denote the decryption scheme corresponding to E_2 . For confidential data $x, y \in \mathbb{R}$ and binary operator $\star \in \{+, \times\}$, a task can be formulated as $x \star y = D_2(LLM(E_2(x), E_2(y), \star))$.

We report the evaluation results for FHE agents in Table II. Our evaluation results on addition and multiplication suggest that this defense is effective for AI agents requiring calculations on sensitive data supported by these operations. Thus, FHE is a solution for maintaining privacy during operations on sensitive data. Overall, our encryption defense does not substantially compromise the usability of AI agents and highlights a potential direction for future research on privacy-preserving AI agents.

2) *Session-aware models for AI agents:* An alternative to sessionless defenses is to make session-aware AI models. Towards this direction, OpenAI recently introduced Temporary Chat ², where they promised not to use the chat history to improve their models. However, not improving the model on agent tasks would limit agent intelligence and user experience. To build powerful agent programs to handle diverse tasks, learning actions are essential.

One approach to privacy-preserving AI agents with personalization is fine-tuning each user’s LLM on their own chat history, isolating model updates per user. However, this is costly and limited by available data. Alternatives like in-context learning [44] and retrieval-augmented generation [45] enhance responses by embedding past contexts in prompts, but are constrained by the length of model’s context window. A more promising method is prompt tuning [46], which freezes the foundational model and adds a few user-specific learnable parameters θ_P only to remember chat history. This technique avoids sharing data with the foundation model provider, directly addressing privacy concerns.

V. RELATED WORK

Recent advancements in LLMs have had a significant impact in the development of AI agent, particularly in their ability to reason based on natural language prompts to observe and interact with their environments dynamically [20, 25]. This shift from reinforcement learning to LLM agents has ushered in a new wave of AI agent development, where the emphasis is on enabling agents to perform actions based on natural language commands. ReAct [23] introduced chain-of-thought prompting [20] to guide pre-trained LLMs

²<https://help.openai.com/en/articles/8914046-temporary-chat-faq>

to follow instructions in the agent setting. This approach has since been applied to computer tasks [22] and other real-world tasks [3, 6, 47, 48]. To evaluate the performance of the agents, several benchmarks [4, 5] have been proposed. These benchmarks measure the correctness of an agent’s actions without considering the potential vulnerabilities that agent actions can cause to the environment.

The threats to LLMs and AI agents are different [49]. For LLMs, the concerns primarily address model alignment with human values, including ethics, offensive language, and politics [29]. Conversely, AI agents, which use LLMs to generate actions and access tools, pose threats to real computing systems, applications, and resources, compromising their confidentiality, integrity, and availability.

VI. CONCLUSION

With the aid of tool-augmented LLMs, AI agents are being recognized as a promising direction toward artificial assistants. Considerable research has focused on enhancing the accuracy of AI agent actions through advanced reasoning, planning, and learning. However, despite high performance in controlled evaluation settings, the potential side effects and dangers posed by these methods have not been thoroughly examined. In this paper, we present a systematic analysis of the security issues in current AI agent development and propose practical and feasible defense strategies. We discuss the potential vulnerabilities of AI agents both theoretically and in realistic scenarios with security-centric examples, and propose multiple defense techniques for each identified vulnerability. We highlight the future research directions and best practices for developing secure agent programs, and believe our work could boost the advancement of secure and trustworthy AI agents. Our code and data are publicly available ³.

ACKNOWLEDGEMENT

This work is partially supported by UC Noyce Initiative.

REFERENCES

- [1] David E Wilkins. *Practical planning: extending the classical AI planning paradigm*. Elsevier, 2014.
- [2] Charles Isbell, Christian R Shelton, Michael Kearns, Satinder Singh, and Peter Stone. “A social reinforcement learning agent”. In: *International conference on Autonomous agents*. 2001.
- [3] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. “Webshop: Towards scalable real-world web interaction with grounded language agents”. In: *Advances in Neural Information Processing Systems* (2022).
- [4] Xiao Liu et al. “AgentBench: Evaluating LLMs as Agents”. In: *International Conference on Learning Representations*. 2024.
- [5] Shuyan Zhou et al. “WebArena: A Realistic Web Environment for Building Autonomous Agents”. In: *International Conference on Learning Representations*. 2024.
- [6] Joon Sung Park et al. “Generative agents: Interactive simulacra of human behavior”. In: *Annual acm symposium on user interface software and technology*. 2023.
- [7] Matt Bishop. *Introduction to computer security*. Addison-Wesley Professional, 2004.
- [8] Nicholas Carlini et al. “Quantifying Memorization Across Neural Language Models”. In: *International Conference on Learning Representations*. 2023.
- [9] Kushal Tirumala, Aram Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. “Memorization without overfitting: Analyzing the training dynamics of large language models”. In: *Advances in Neural Information Processing Systems* (2022).
- [10] Gregoire Deletang et al. “Language Modeling Is Compression”. In: *The Twelfth International Conference on Learning Representations*. 2024.
- [11] Theodore Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas Griffiths. “Cognitive Architectures for Language Agents”. In: *Transactions on Machine Learning Research* (2024).
- [12] Jacob Steinhardt, Pang Wei W Koh, and Percy S Liang. “Certified defenses for data poisoning attacks”. In: *Neural information processing systems* (2017).
- [13] Keita Kurita, Paul Michel, and Graham Neubig. “Weight Poisoning Attacks on Pretrained Models”. In: *Annual Meeting of the Association for Computational Linguistics*. 2020.
- [14] Shuli Jiang, Swanand Kadhe, Yi Zhou, Ling Cai, and Nathalie Baracaldo. “Forcing Generative Models to Degenerate Ones: The Power of Data Poisoning Attacks”. In: *NeurIPS Workshop on Backdoors in Deep Learning-The Good, the Bad, and the Ugly*. 2023.
- [15] Siladitya Ray. “Samsung bans ChatGPT among employees after sensitive code leak”. In: *Forbes* (May 2023). URL: <https://www.forbes.com/sites/siladityaray/2023/05/02/samsung-bans-chatgpt-and-other-chatbots-for-employees-after-sensitive-code-leak/?sh=64cca5116078>.
- [16] Nicholas Carlini et al. “Extracting training data from large language models”. In: *30th USENIX Security Symposium (USENIX Security 21)*. 2021.
- [17] Xueluan Gong, Yanjiao Chen, Wenbin Yang, Guanghao Mei, and Qian Wang. “InverseNet: Augmenting Model Extraction Attacks with Training Data Inversion.” In: *IJCAI*. 2021, pp. 2439–2447.
- [18] Anthony Brohan et al. “Do as i can, not as i say: Grounding language in robotic affordances”. In: *Conference on robot learning*. PMLR. 2023, pp. 287–318.
- [19] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. “Language models as zero-shot planners: Extracting actionable knowledge for embodied

³<https://github.com/SecurityLab-UCD/ai-agent-security>

- agents". In: *International Conference on Machine Learning*. PMLR. 2022, pp. 9118–9147.
- [20] Jason Wei et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in neural information processing systems* (2022).
- [21] Xuezhi Wang et al. "Self-Consistency Improves Chain of Thought Reasoning in Language Models". In: *International Conference on Learning Representations*. 2022.
- [22] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. "Language models can solve computer tasks". In: *Advances in Neural Information Processing Systems* (2024).
- [23] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations*. 2023.
- [24] Shibo Hao et al. "Reasoning with Language Model is Planning with World Model". In: *Conference on Empirical Methods in Natural Language Processing*. 2023.
- [25] Shunyu Yao et al. "Tree of thoughts: Deliberate problem solving with large language models". In: *Advances in Neural Information Processing Systems* (2024).
- [26] Yuchen Zhuang et al. "ToolChain*: Efficient Action Space Navigation in Large Language Models with A* Search". In: *International Conference on Learning Representations*. 2024.
- [27] Yinger Zhang et al. "Reverse Chain: A Generic-Rule for LLMs to Master Multi-API Planning". In: *Findings of the Association for Computational Linguistics: NAACL*. 2024.
- [28] Fábio Perez and Ian Ribeiro. "Ignore Previous Prompt: Attack Techniques For Language Models". In: *NeurIPS ML Safety Workshop*. 2022.
- [29] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. "LLM-Fuzzer: Scaling Assessment of Large Language Model Jailbreaks". In: *USENIX Security*. 2024.
- [30] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. "StruQ: Defending Against Prompt Injection with Structured Queries". In: *USENIX Security*. 2024.
- [31] Wenlong Huang et al. "Inner Monologue: Embodied Reasoning through Planning with Language Models". In: *Conference on Robot Learning*. PMLR. 2023.
- [32] Clayton Allen Davis, Onur Varol, Emilio Ferrara, Alessandro Flammini, and Filippo Menczer. "Botornot: A system to evaluate social bots". In: *International conference companion on world wide web*. 2016.
- [33] John Launchbury and Simon L Peyton Jones. "State in haskell". In: *Lisp and symbolic computation* (1995).
- [34] Mark P Jones and Luc Duponcheel. *Composing monads*. Tech. rep. Technical Report YALEU/DCS/RR-1004, Department of Computer Science. Yale ..., 1993.
- [35] Cole Schlesinger and Nikhil Swamy. *Verification condition generation with the Dijkstra state monad*. Tech. rep. Technical Report MSR-TR-2012-45, 2012.
- [36] Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. "Verifying higher-order programs with the Dijkstra monad". In: *ACM SIGPLAN Notices* 48.6 (2013), pp. 387–398.
- [37] Simon Gay and Malcolm Hole. "Types and subtypes for client-server interactions". In: *Programming Languages and Systems: 8th European Symposium on Programming, ESOP'99*. Springer. 1999.
- [38] Daniel Giffin et al. "Hails: Protecting data privacy in untrusted web applications". In: *Journal of Computer Security* 25.4-5 (2017), pp. 427–461.
- [39] David Cock, Gerwin Klein, and Thomas Sewell. "Secure microkernels, state monads and scalable refinement". In: *Theorem Proving in Higher Order Logics: International Conference*. Springer. 2008.
- [40] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.
- [41] Mihir Bellare, Phillip Rogaway, and Terence Spies. "The FFX mode of operation for format-preserving encryption". In: *NIST submission* (2010).
- [42] Michael Naehrig, Kristin Lauter, and Vinod Vaikuntanathan. "Can homomorphic encryption be practical?" In: *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*. 2011.
- [43] Abbas Acar, Hidayet Aksu, A Selcuk Uluagac, and Mauro Conti. "A survey on homomorphic encryption schemes: Theory and implementation". In: *ACM Computing Surveys (Csur)* 51.4 (2018), pp. 1–35.
- [44] Tom Brown et al. "Language models are few-shot learners". In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [45] Patrick Lewis et al. "Retrieval-augmented generation for knowledge-intensive nlp tasks". In: *Advances in Neural Information Processing Systems* (2020).
- [46] Brian Lester, Rami Al-Rfou, and Noah Constant. "The Power of Scale for Parameter-Efficient Prompt Tuning". In: *Conference on Empirical Methods in Natural Language Processing*. 2021.
- [47] Zihao Wang et al. "Describe, explain, plan and select: interactive planning with large language models enables open-world multi-task agents". In: *International Conference on Neural Information Processing Systems*. 2023, pp. 34153–34189.
- [48] Yu Gu, Xiang Deng, and Yu Su. "Don't Generate, Discriminate: A Proposal for Grounding Language Models to Real-World Environments". In: *Annual Meeting of the Association for Computational Linguistics*. 2023.
- [49] Zehang Deng et al. *AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways*. 2024. arXiv: 2406.02630 [cs.CR].